

Java DSA — 100 Question Practice Set

Phase 1 → Phase 2 → Phase 3 | Placement Preparation

A progressive question set designed to build DSA fundamentals, core problem-solving patterns, and advanced placement skills through continuous practice.

Phase 1 — Foundations (Questions 1–25)

1. **[Foundations]** Find the largest and smallest element in an integer array without using built-in min/max functions.
2. **[Foundations]** Calculate the sum and average of all elements in an array.
3. **[Foundations]** Count positive, negative, even, and odd numbers in an array.
4. **[Arrays]** Reverse an array in-place using $O(1)$ extra space.
5. **[Arrays]** Check whether an array is sorted in non-decreasing order.
6. **[Arrays]** Find the second largest distinct element in an array.
7. **[Arrays]** Remove duplicates from a sorted array in-place.
8. **[Arrays]** Move all zeroes to the end while preserving the order of non-zero elements.
9. **[Searching]** Implement linear search and return the index of the target.
10. **[Searching]** Implement binary search on a sorted array.
11. **[Searching]** Find the first and last occurrence of a target in a sorted array.
12. **[Searching]** Count how many times a target occurs in a sorted array.
13. **[Strings]** Reverse a String without using a built-in reverse function.
14. **[Strings]** Check whether a String is a palindrome.
15. **[Strings]** Count vowels, consonants, digits, spaces, and special characters.
16. **[Strings]** Check whether two Strings are anagrams.
17. **[Strings]** Find the first non-repeating character in a String.
18. **[Hashing]** Count the frequency of every element in an integer array.
19. **[Hashing]** Find the element that appears once when every other element appears twice.
20. **[Math]** Determine whether a number is prime and print all prime numbers up to N .
21. **[Recursion]** Calculate factorial using recursion and state its time and space complexity.
22. **[Recursion]** Generate the first N Fibonacci numbers using recursion, then identify its performance issue.
23. **[Recursion]** Find the sum of digits of a number using recursion.
24. **[Complexity]** For a given Java loop containing nested loops and logarithmic iteration, determine its Big-O time complexity.
25. **[Sorting]** Implement insertion sort and test it on sorted, reverse-sorted, and random arrays.

Phase 2 — Core Problem Solving (Questions 26–60)

26. **[Arrays + Hashing]** Find two indices whose values add up to a given target.
27. **[Arrays + Hashing]** Find all duplicate elements in an integer array using a HashSet.
28. **[Arrays + Hashing]** Find the majority element appearing more than $N/2$ times.
29. **[Prefix Sum]** Find the equilibrium index where the sum on the left equals the sum on the right.
30. **[Prefix Sum]** Answer multiple range-sum queries on an array efficiently.
31. **[Two Pointers]** Find a pair with a given sum in a sorted array using two pointers.
32. **[Two Pointers]** Remove duplicates from a sorted array using the two-pointer technique.
33. **[Two Pointers]** Find the intersection of two sorted arrays without duplicate output.
34. **[Sliding Window]** Find the maximum sum of any subarray of fixed size K .
35. **[Sliding Window]** Find the longest substring without repeating characters.
36. **[Sliding Window]** Find the smallest subarray with sum at least K when all elements are positive.
37. **[Kadane]** Find the maximum sum contiguous subarray and return its maximum sum.
38. **[Arrays]** Rotate an array to the right by K positions in $O(N)$ time and $O(1)$ extra space.
39. **[Arrays]** Find the missing number from an array containing values from 0 to N .
40. **[Arrays]** Find the maximum profit from one buy and one sell of stock prices.
41. **[Sorting]** Implement merge sort and explain why its worst-case complexity is $O(N \log N)$.
42. **[Sorting]** Implement quicksort and handle duplicate values efficiently.
43. **[Intervals]** Merge all overlapping intervals.
44. **[Intervals]** Insert a new interval into a sorted list of non-overlapping intervals and merge if necessary.
45. **[Linked List]** Implement a singly linked list with insertion at beginning, end, and a given position.
46. **[Linked List]** Delete a node by value from a singly linked list.
47. **[Linked List]** Reverse a singly linked list iteratively.
48. **[Linked List]** Find the middle node of a linked list using slow and fast pointers.
49. **[Linked List]** Detect whether a linked list contains a cycle using $O(1)$ extra space.
50. **[Linked List]** Merge two sorted linked lists into one sorted linked list.
51. **[Stack]** Implement a stack using an array with push, pop, peek, and isEmpty.
52. **[Stack]** Check whether brackets in an expression are balanced.
53. **[Stack]** Evaluate a postfix expression using a stack.
54. **[Stack]** Find the next greater element for every element in an array.
55. **[Queue]** Implement a queue using two stacks.
56. **[Heap]** Find the K largest elements in an array using a PriorityQueue.
57. **[Heap]** Find the K -th largest element in an unsorted array using a heap.

58. **[Binary Tree]** Implement preorder, inorder, and postorder traversal of a binary tree.
59. **[Binary Tree]** Perform level-order traversal of a binary tree.
60. **[Binary Tree]** Find the height and maximum depth of a binary tree.

Phase 3 — Advanced Problem Solving (Questions 61–100)

61. **[Trees]** Check whether a binary tree is height-balanced.
62. **[Trees]** Find the diameter of a binary tree.
63. **[Trees]** Check whether two binary trees are identical.
64. **[Trees]** Find the Lowest Common Ancestor of two nodes in a binary tree.
65. **[BST]** Implement insertion, search, and deletion in a Binary Search Tree.
66. **[BST]** Validate whether a binary tree is a valid Binary Search Tree.
67. **[BST]** Find the K-th smallest element in a Binary Search Tree.
68. **[Trees]** Print the top view of a binary tree.
69. **[Graph]** Represent a graph using an adjacency list and perform BFS.
70. **[Graph]** Perform DFS on a graph using both recursion and iteration.
71. **[Graph]** Count the number of connected components in an undirected graph.
72. **[Graph]** Detect a cycle in an undirected graph using BFS or DFS.
73. **[Graph]** Detect a cycle in a directed graph.
74. **[Graph]** Check whether an undirected graph is bipartite.
75. **[Graph]** Find the shortest path between two vertices in an unweighted graph.
76. **[Graph]** Perform topological sorting of a directed acyclic graph using Kahn's algorithm.
77. **[Graph]** Perform topological sorting using DFS and compare it with Kahn's algorithm.
78. **[Backtracking]** Generate all subsets of an array using recursion/backtracking.
79. **[Backtracking]** Generate all permutations of an array containing distinct elements.
80. **[Backtracking]** Generate all combinations of K numbers chosen from 1 to N.
81. **[Backtracking]** Solve the N-Queens problem.
82. **[Greedy]** Schedule the maximum number of non-overlapping activities given their start and finish times.
83. **[Greedy]** Given coin denominations and an amount, determine the minimum number of coins using a greedy approach and identify when the approach fails.
84. **[Greedy]** Find the minimum number of platforms required for trains given arrival and departure times.
85. **[Dynamic Programming]** Solve the climbing stairs problem using recursion, memoization, and tabulation.
86. **[Dynamic Programming]** Find the maximum sum of non-adjacent elements in an array.
87. **[Dynamic Programming]** Solve the 0/1 Knapsack problem using dynamic programming.
88. **[Dynamic Programming]** Find the minimum number of coins required to make a target amount.
89. **[Dynamic Programming]** Find the length of the Longest Common Subsequence of two Strings.
90. **[Dynamic Programming]** Find the Longest Increasing Subsequence of an integer array.
91. **[Trie]** Implement a Trie with insert, search, and startsWith operations.

92. **[Monotonic Stack]** Find the largest rectangle area in a histogram.
93. **[Graph]** Implement Dijkstra's shortest-path algorithm using Java PriorityQueue.
94. **[Graph + DSU]** Implement Disjoint Set Union with path compression and union by rank/size.
95. **[Graph + DSU]** Find the Minimum Spanning Tree of a weighted graph using Kruskal's algorithm.
96. **[Mixed]** Find the longest subarray with sum K when the array can contain positive and negative numbers.
97. **[Mixed]** Find the length of the longest consecutive sequence in an unsorted array in $O(N)$ expected time.
98. **[Mixed]** Find all unique triplets in an array whose sum is zero.
99. **[Mixed]** Design an LRU Cache supporting get and put in $O(1)$ average time.
100. **[Placement Mix]** Given an unseen array/string problem, identify the appropriate pattern, solve it in Java, and state its time and space complexity.